



UNIVERSITY OF TEHRAN
Electrical and Computer Engineering Department
Digital Logic Design, ECE 367 / Digital Systems I, ECE 894
Spring 1402-03
Computer Assignment 5
State Machines and Basic RTL

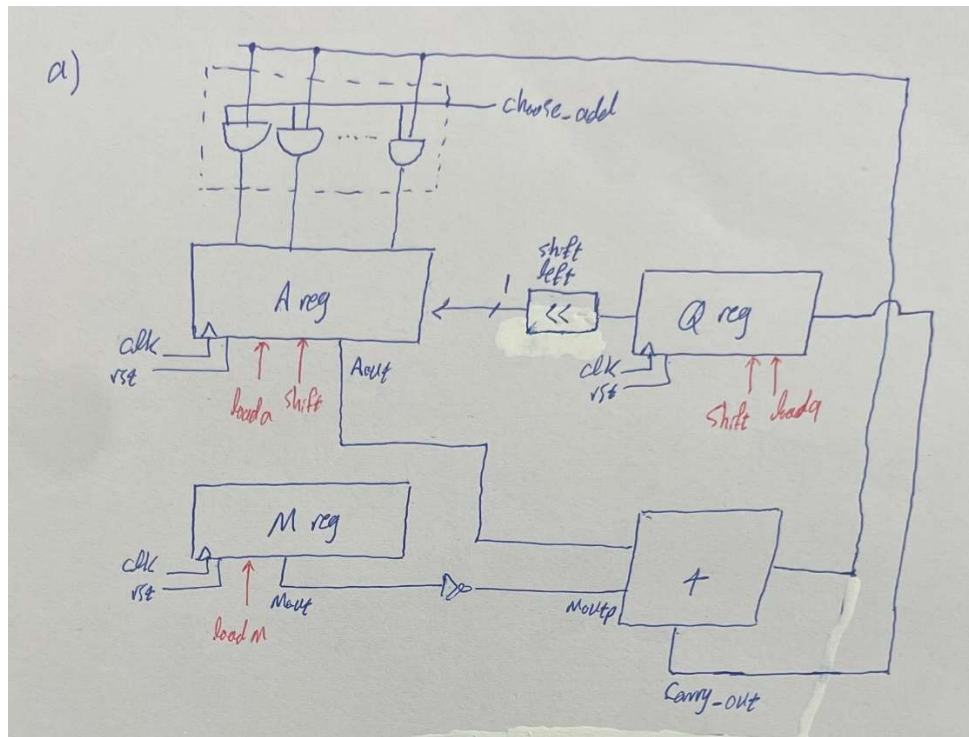
Student Name: Matin Shiasi

Student ID: 810101453

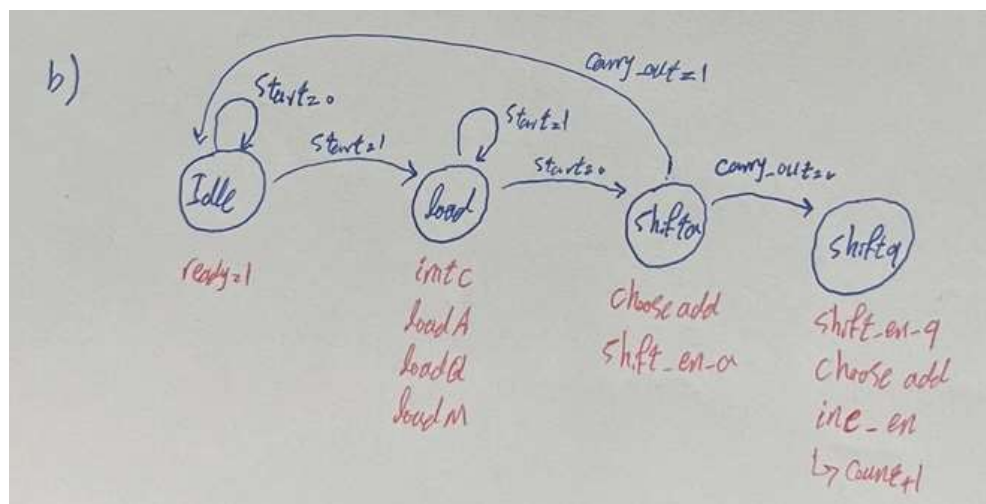
Due Date: 1403/3/20

Q1)

a)



b)



c)

```

1 module Datapath #(parameter DATA_WIDTH = 5, parameter DIVISOR = 5) (
2   input wire clk, reset, load_ena, load_enm, shift_ena, chooseadd,
3   input wire [DATA_WIDTH-1:0] qin,
4   input wire [DIVISOR-1:0] min,
5   output wire [DIVISOR-2:0] quotient_dp_out,
6   output wire [DATA_WIDTH-1:0] remainder_dp_out,
7   output wire cout
8 );
9
10 wire [DIVISOR-1:0] mout;
11 wire [DIVISOR-2:0] qout;
12 wire [DIVISOR-1:0] moutp;
13 wire [DATA_WIDTH-1:0] aout;
14 wire [DATA_WIDTH-1:0] address;
15 wire [DATA_WIDTH-1:0] ainput;
16
17 assign moutp = ~mout;
18 assign {cout, address} = moutp + aout + 5'b00001;
19 assign remainder_dp_out = aout;
20 assign quotient_dp_out = qout;
21 assign ainput = chooseadd ? address : 5'b0;
22
23 shift_register #(N(5)) a (
24   .clk(clk),
25   .reset(reset),
26   .load_en(load_ena),
27   .serial_in(qout[1]),
28   .shift_en(shift_ena),
29   .parallel_in(ainput),
30   .q(aout)
31 );
32 load_register #(N(5)) m (
33   .clk(clk),
34   .reset(reset),
35   .load_en(load_enm),
36   .d(min),
37   .q(mout)
38 );
39 shift_register #(N(4)) q (
40   .clk(clk),
41   .reset(reset),
42   .shift_en(shift_ena),
43   .serial_in(cout),
44   .q(qout),
45   .parallel_in(qin),
46   .load_en(load_ena)
47 );
48
49 endmodule

```

```

1 module load_register #(parameter N = 8) (
2   input wire clk, reset, load_en,
3   input wire [N-1:0] d,
4   output reg [N-1:0] q
5 );
6
7 always @(posedge clk or posedge reset) begin
8   if (reset) begin
9     q <= 0;
10  end else if (load_en) begin
11    q <= d;
12  end
13 endmodule
14
15 module shift_register #(parameter N = 8) (
16   input wire serial_in, clk, reset, load_en, shift_en,
17   input wire [N-1:0] parallel_in,
18   output reg [N-1:0] q
19 );
20
21 always @(posedge clk or posedge reset) begin
22   if (reset) begin
23     q <= 0;
24   end else if (shift_en) begin
25     q <= {q[N-2:0], serial_in};
26   end else if (load_en) begin
27     q <= parallel_in;
28   end
29 endmodule

```

طبق طراحی مدار پیش میرویم. ابتدا مقدار دهی های اولیه را انجام می دهیم و سپس خروجی رجیستر a را از خروجی رجیستر m کم می کنیم که حاصل این تفریق در صورتی که chooseadd برابر یک باشد وارد یک سری and دو به دو میشوند و جواب این تفریق را در A نگه می دارد. در رجیستر Q ابتدا آن را شیفت می دهیم و جای خالی را با توجه به cout پر می کنیم و و بیت شیفت داده شه را به عنوان serial-in به رجیستر a می دهیم. به همین علت است که تعداد بیت های رجیستر a را یک واحد بیشتر از رجیستر q در نظر گرفتیم. در نهایت این فرایند تا موقعی که cout برابر 7 شود ادامه می دهیم.

دو ماژول جدا به نام load_register و shift_register داریم که اولی فرایند لود کردن و دومی در صورت فعال بودن حالت شیفت، فرایند وارد شدن serial-in و شیفت دادن آن به همان اندازه و در صورت فعال بودن حالت لود ورودی را در خروجی لود می کند.

(d)

در بخش کنترلر کد ابتدا state ها را تعریف می کنیم. جای متغیر n از counter استفاده کردیم که با initc ابتدا آن را صفر مقدار دهی می کنیم و در هر مرحله از load-state با هر بار شیفت یافتن q و فعال بودن incen آن را یک واحد زیاد می کنیم. در مرحله بعدی که shiftingq-state است تصمیم می گیریم که اگر حاصل تفریق منفی شد، خود A را به عنوان باقیمانده در نظر بگیریم و برای این کار باید loada مربوط به آن را فعال کنیم تا همان مقدار لود شود و تغییری نبینیم اما اگر حاصل تفریق مثبت شد و A را لود نمی کنیم چرا که هنوز به باقیمانده درست نرسیده ایم.

```

1  module Controller #(parameter DATA_WIDTH = 5, parameter DIVISOR = 5) (
2      input clk, reset, start, cout,
3      output reg chooseadd, shift_enq, shift_ena, load_enm, load_enq, load_ena, ready
4  );
5
6      reg incen, initinc;
7      reg [1:0] ps, ns;
8      reg [1:0] count;
9      reg carry_out = 0;
10
11      parameter [2:0]
12          idle = 0,
13          loadings = 1,
14          shiftinga = 2,
15          shiftingq = 3,
16          finalloading = 4;
17
18      always @(posedge clk or posedge reset) begin
19          if (reset) begin
20              ps <= idle;
21          end else begin
22              ps <= ns;
23          end
24      end
25
26      always @(posedge clk or posedge reset) begin
27          if (reset) begin
28              count <= 0;
29          end else if (initinc) begin
30              count <= 0;
31          end else if (incen) begin
32              count <= count + 1;
33          end
34          carry_out <= &count;
35      end
36
37      always @(ps, start, carry_out, cout) begin
38          {incen, chooseadd, shift_enq, shift_ena, load_enm, load_enq, ready, load_ena} = 8'd0;
39
40          case (ps)
41              idle: begin
42                  ready = 1'b1;
43                  ns = start ? loadings : idle;
44              end
45              loadings: begin
46                  ns = start ? loadings : shiftinga;
47                  initinc = 1'b1;
48                  load_enm = 1'b1;
49                  load_enq = 1'b1;
50                  load_ena = 1'b1;
51                  chooseadd = 1'b0;
52                  ready = 0;
53              end
54              shiftinga: begin
55                  ns = carry_out ? idle : shiftingq;
56                  {load_enq, load_enm} = 2'b00;
57                  {incen, initinc} = 2'b00;
58                  {shift_enq, shift_ena, chooseadd} = 3'b011;
59              end
60              shiftingq: begin
61                  ns = shiftinga;
62                  incen = 1;
63                  {shift_enq, shift_ena, chooseadd} = 3'b101;
64                  load_ena = cout;
65              end
66          endcase
67      end
68  endmodule

```

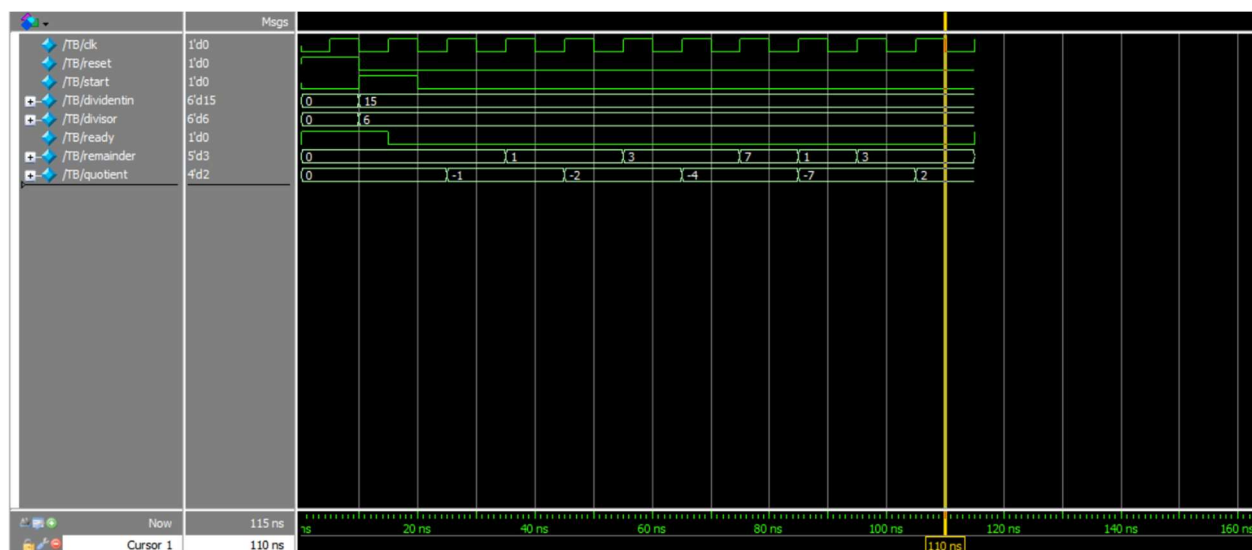
```

1  module Top_Module #(parameter DATA_WIDTH = 5, parameter DIVISOR = 5) (
2      input clk, rst, start,
3      input [DATA_WIDTH:0] dividentin,
4      input [DIVISOR:0] divisor,
5      output ready,
6      output [DATA_WIDTH-1:0] remainder,
7      output [DATA_WIDTH-2:0] quotient
8  );
9
10     wire chooseadd, shift_enq, shift_ena, load_enm, load_ena, load_enq, cout;
11
12     Datapath #(DATA_WIDTH, DIVISOR) datapath_inst (
13         .clk(clk), .reset(rst), .quotient_dp_out(quotient), .remainder_dp_out(remainder),
14         .qin(dividentin), .load_enq(load_enq), .load_enm(load_enm), .shift_enq(shift_enq),
15         .shift_ena(shift_ena), .load_ena(load_ena), .chooseadd(chooseadd), .min(divisor),
16         .cout(cout)
17     );
18
19     Controller #(DATA_WIDTH, DIVISOR) controller_inst (
20         .clk(clk), .reset(rst), .start(start), .cout(cout), .chooseadd(chooseadd),
21         .load_enq(load_enq), .load_ena(load_ena), .shift_enq(shift_enq), .shift_ena(shift_ena),
22         .load_enm(load_enm), .ready(ready)
23     );
24
25 endmodule

```

e & f)

در تست پنج داده شده هم عدد 15 را بر 6 تقسیم کرده ایم که مقدار خارج قسمت و باقیمانده را به درستی نشان میدهد.



(g)

در این بخش کد را به کوارتز داده تا سنتز شده آن را بگیریم. (فایل VO در پیوست است) و خروجی های کوارتز به این صورت است.

Flow Status	Successful - Sun Jun 09 17:51:18 2024
Quartus Prime Version	20.1.0 Build 711 06/05/2020 SJ Lite Edition
Revision Name	ca5
Top-level Entity Name	Top_Module
Family	Cyclone IV E
Device	EP4CE115F29C8
Timing Models	Final
Total logic elements	25 / 114,480 (< 1 %)
Total registers	20
Total pins	25 / 529 (5 %)
Total virtual pins	0
Total memory bits	0 / 3,981,312 (0 %)
Embedded Multiplier 9-bit elements	0 / 532 (0 %)
Total PLLs	0 / 4 (0 %)